`leakcheck`: screening reported classification results for the resampling-before-splitting artefact

Author names withheld for review
Affiliations withheld for review

Abstract

`leakcheck` estimates whether a published classification result resembles the statistical pattern produced, in a controlled corpus, by applying a synthetic over-sampler before splitting a dataset into training and test partitions rather than to the training partition alone. It takes only quantities that papers commonly report—the headline minority-class F1, the imbalance ratio, the number of minority cases, the number of features and the model family—and returns a model-estimated screening probability together with a short written explanation. The screening model is a gradient-boosted classifier trained on 22,400 labelled results from 41 public datasets in which the same models were evaluated under both orderings, so the experimental condition is known by construction; leave-one-dataset-out validation gives an AUC of 0.795 and a Brier score of 0.197. The output is a triage signal rather than a causal finding of leakage. The trained model ships as plain JSON and is evaluated by a pure-NumPy tree walker, limiting run-time dependencies and reducing sensitivity to changes in modelling-library serialization.

Keywords: data leakage; class imbalance; SMOTE; peer review; research software; reproducibility

1 Summary

Applying SMOTE [1] or a related over-sampler to a whole dataset and only then splitting it into training and test partitions is a documented form of data leakage in applied machine learning [2, 3, 4, 46]. Because synthetic observations may then be constructed using information from cases assigned to the test partition, the resulting score can overestimate performance on genuinely unseen cases.

That error sits inside a broader reproducibility problem. Consensus recommendations, community surveys and practical guides continue to document leakage, non-replicable benchmarks and overstated conclusions in applied machine learning [47, 48, 49, 6, 8]. `leakcheck` is a small Python package that screens a reported result for one identifiable signature of that problem. It is designed for readers rather than authors: it requires no access to code or data, but relies on the relevant quantities being reported clearly in a methods section or results table.

2 Statement of need

Reviewers and editors may lack the time, access, or computational environment needed to run an author's code. Existing guidance is often qualitative and may not resolve questions about a specific reported result. `leakcheck` turns the general warning into a question about the case in hand: given what this paper reports, how unusual is this number relative to the controlled corpus? Figure 1 is the path from a PDF to that question.

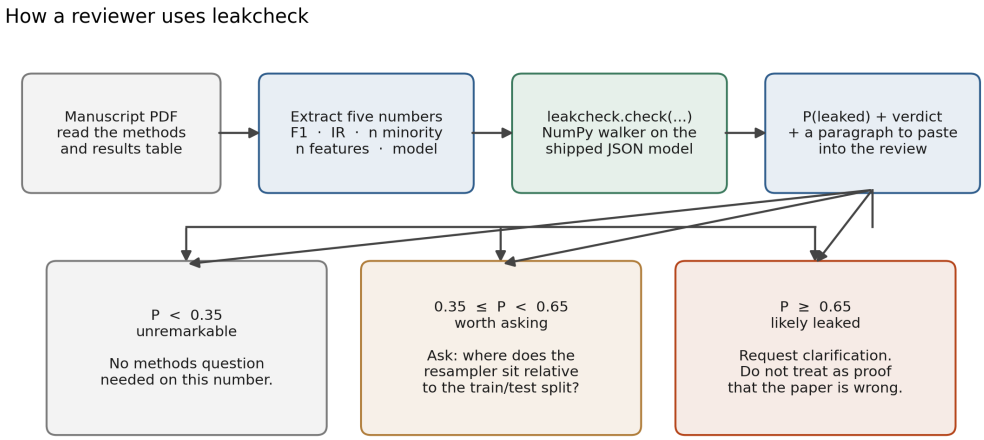


Figure 1: How a reviewer or editor uses `leakcheck`. Five quantities are copied from the manuscript; the package returns a probability, a verdict band, and a paragraph suitable for a review. The strongest recommended action is to ask where the resampler sits relative to the split.

The empirical basis is a regularity observed in the companion study. Within that corpus, train-only scores varied more strongly with problem difficulty, whereas resample-then-split scores were more concentrated near the upper end of the observed range. The variance of the train-only scores was 4.86 times that of the leaked scores. Figure 2 shows kernel density estimates for the 11,200 matched comparisons used to train the screen. This association with resampling order is consistent with the wider class-imbalance literature, including theoretical and simulation work on SMOTE and on the possible harms of imbalance corrections [14, 18, 25, 50, 51, 52].

- **Peer review.** Decide whether to ask the authors where the resampler sits in their pipeline. The tool prints a paragraph suitable for pasting into a report, which states that the output is a screen rather than a finding.
- **Self-audit.** Check one's own results before submission.
- **Meta-research.** Screen a corpus of published results to estimate how widespread the pattern is in a field; a single command accepts a table.

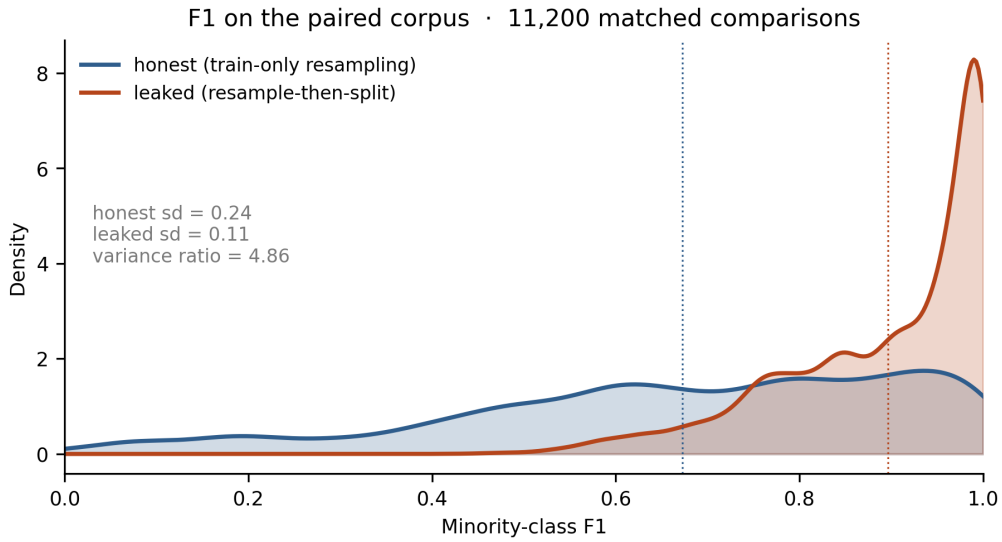


Figure 2: Minority-class F1 under the two orderings in 11,200 matched comparisons. Train-only scores are more dispersed, while resample-then-split scores are more concentrated near the upper end of the observed range. The train-only distribution has 4.86 times the variance of the resample-then-split distribution.

3 How the screening model was built

Each matched pair in the companion corpus yields two labelled examples: a resample-then-split result and a train-only result, matched on the quantities a paper would report except for the resulting score. A gradient-boosted classifier is trained to separate them from the reported F1, the log imbalance ratio, the log minority count, the log feature count and the model family. Validation is leave-one-dataset-out, so each held-out prediction comes from a model fitted without that dataset. This design reduces, but does not eliminate, the risk that performance reflects memorisation of dataset-specific patterns.

3.1 Training corpus and experimental design

The controlled corpus contains 11,200 matched comparisons and therefore 22,400 labelled records: one record from the resample-then-split ordering and one from the train-only ordering for each comparison. Within a pair, the source data, preprocessing, classifier family, resampler and experimental configuration are held fixed; only the position of resampling relative to the train/test split changes. This pairing makes the experimental condition known by construction and limits confounding by between-dataset difficulty within each comparison.

The 41 source datasets span 208 to 5,000 rows, 3 to 180 features, 30 to 1,813 minority cases, and imbalance ratios from 1.02 to 109.94. Six datasets concern employee attrition; the remaining 35 are drawn from the Penn Machine Learning Benchmarks [65]. The application mix comprises biology (3 datasets), census (1), chemistry (2), computer vision (6), demography (2), finance (3),

games (2), human resources (6), marketing (1), medicine (8), physics (3), speech (2), and text (2). Table 1 summarises the fitted domain and Appendix A provides the dataset-level inventory. The archive identified under Data Availability provides the released model, fixtures and supporting corpus documentation.

The reported AUC is 0.795, with accuracy 0.697 at the natural threshold and a Brier score of 0.197 [66]. Per-dataset AUC has median 0.987 and minimum 0.388, indicating substantial heterogeneity across datasets. An ablation that removes the reported F1 and leaves only the dataset descriptors falls to 0.500, close to chance; within this corpus, that result is consistent with the signal depending on the relationship between the score and the dataset descriptors rather than on the descriptors alone. Table 1 summarises the fitted model; Figure 3 shows the leave-one-dataset-out ROC and example screening curves. Figure 4 is the matching reliability diagram. In the held-out predictions, observed frequencies align more closely with predicted probabilities in the tails than in the middle of the range. The package therefore reports verdict bands and does not present the numerical output as a precisely calibrated probability for an individual paper.

The displayed value should consequently be read as a screening probability under the controlled training distribution, not as the posterior probability that a particular paper committed an error. The bands below 0.35, from 0.35 to 0.65, and above 0.65 are operational triage categories. Their positive predictive value in a literature review will depend on the unknown prevalence of the error in that literature and on how closely the reviewed cases resemble the training corpus.

Table 1: Provenance of the screening model shipped as JSON with `leakcheck`. Ranges are those of the training corpus; inputs outside them still return a probability, but the result is flagged as an extrapolation.

Item	**Value**
Trees	100
Leave-one-dataset-out AUC	0.795
Datasets	41
Training examples	22,400
Imbalance-ratio range	1.02–109.9
Minority-count range	30–1813
Feature-count range	3–180
Classifier families	5NN, DecTree, GradBoost, LogReg, RandForest, SVM-RBF, XGBoost
Resamplers in the corpus	ADASYN, Borderline, RandomOS, SMOTE
Package version	1.0.0

4 What the screen attends to

The screen is a gradient-boosted tree on five quantities a paper already reports: the headline F1, the log imbalance ratio, the log minority count, the log feature count, and a model-family dummy. Figure 5 shows mean absolute SHAP values of that model on a sample of the paired corpus [34, 53]. In this sample, reported F1 has the largest mean absolute contribution, followed by imbalance and feature count; model-family indicators contribute less on average. These values describe how strongly features affect this fitted model's output and do not establish a causal effect

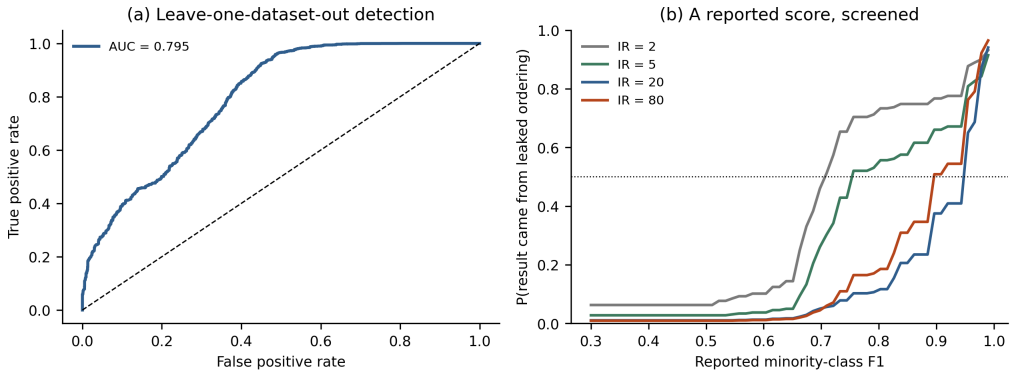


Figure 3: (a) Leave-one-dataset-out ROC for the screen; (b) model-estimated $P(\text{leaked})$ as a function of the reported F1 at four imbalance ratios, with the other inputs fixed at corpus medians.

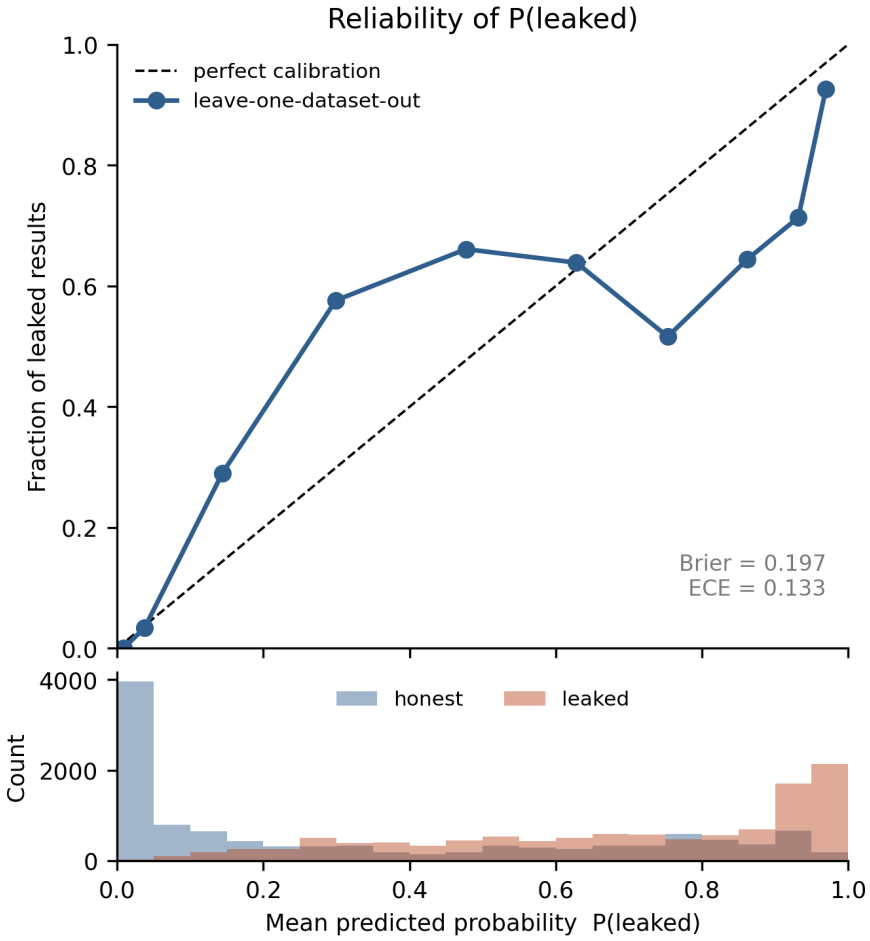


Figure 4: Leave-one-dataset-out reliability diagram of $P(\text{leaked})$. Top: fraction of truly leaked results against mean predicted probability, equal-mass bins, with the diagonal of perfect calibration. Bottom: the distribution of those predictions, split by the true label. Agreement is closest in the tails; the Brier score of the same predictions is 0.197.

or, by themselves, the direction of an effect.

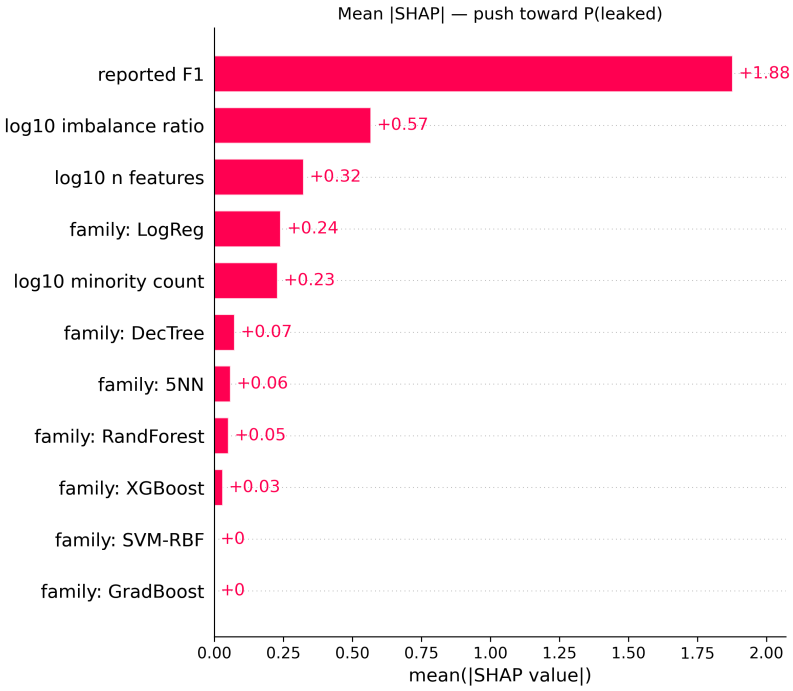


Figure 5: Mean absolute SHAP values of the screening model on the analysed sample. Reported F1 has the largest average absolute contribution; imbalance and feature count contribute less. Mean absolute values quantify contribution magnitude, not causal importance or effect direction.

5 A worked example

A headline F1 of 0.92 on the IBM HR benchmark—237 leavers among 1,470 employees, imbalance 5.2 to 1, 44 encoded features, a random forest—is within the range reported in parts of the attrition literature. Predictive HR analytics raises questions about evaluation quality, evidence, bias, governance, and the limits of algorithmic decisions about people [27, 54, 55, 56, 57]. Because attrition can depend on latent and time-varying organisational factors, a near-perfect score on a small imbalanced benchmark merits scrutiny, although it is not evidence of an error by itself. One study on the IBM table explicitly adopts leakage-free evaluation [58]. Figure 6 treats the example score as an audit trigger: it falls in the package's "likely leaked" band, but the appropriate response is to ask where the resampler sits relative to the split and how the metric was calculated, not to infer that the paper is wrong.

```
import leakcheck
r = leakcheck.check(f1=0.92, imbalance_ratio=5.2, n_minority=237,
                    n_features=44, clf='RandomForest')
print(r.probability, r.verdict)
print(r.explain())
```

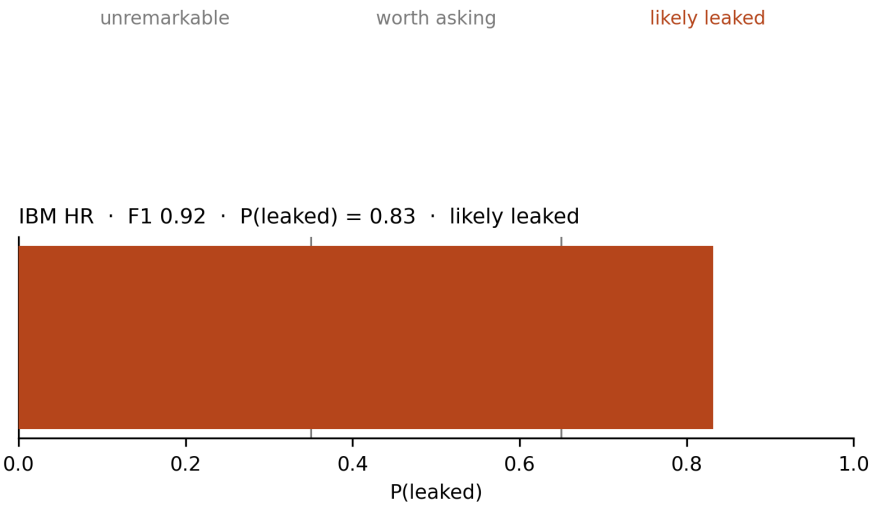


Figure 6: Worked example: IBM HR, reported minority-class F1 0.92, screened with `leakcheck.check`. Vertical lines mark the package thresholds (unremarkable / worth asking / likely leaked).

Table 2 holds minority count, feature count and model family fixed and varies only the reported F1 and the imbalance ratio. Across every row, $P(\text{leaked})$ rises with the score. Figure 7 is the continuous version of that grid, evaluated by the shipped JSON model rather than by a live scikit-learn estimator, so it is what a user of the installed package actually obtains.

Table 2: $P(\text{leaked})$ from `leakcheck.check_many`, RandomForest, 200 minority cases, 30 features. Columns are reported minority-class F1; rows are imbalance ratio. In every row the probability rises with the reported score.

IR	**F1 = 0.45**	**F1 = 0.70**	**F1 = 0.92**
2:1	0.06	0.48	0.74
10:1	0.01	0.05	0.37
50:1	0.01	0.06	0.50

6 Screening published results

`leakcheck` is meant to be pointed at a table of reported results, not only at a single number. Table 3 applies it to headline minority-class F1 values from recent employee-attrition studies on public data [59, 60, 61, 62, 63, 58], using only quantities those papers already report. We classify each study by what it says about its own protocol, never by what the screen guesses. Figure 8 plots the same probabilities, coloured by that stated protocol.

In this small illustrative set, the two studies that describe train-only resampling [63, 58] fall in the unremarkable band. The study that states that resampling preceded the split [61], and several studies that do not state the ordering [59, 60, 62], receive higher screening scores. This pattern motivates a methods question but does not estimate prevalence or establish an error in any individual paper. Alternative explanations—including a macro-averaged F1, different preprocessing, or

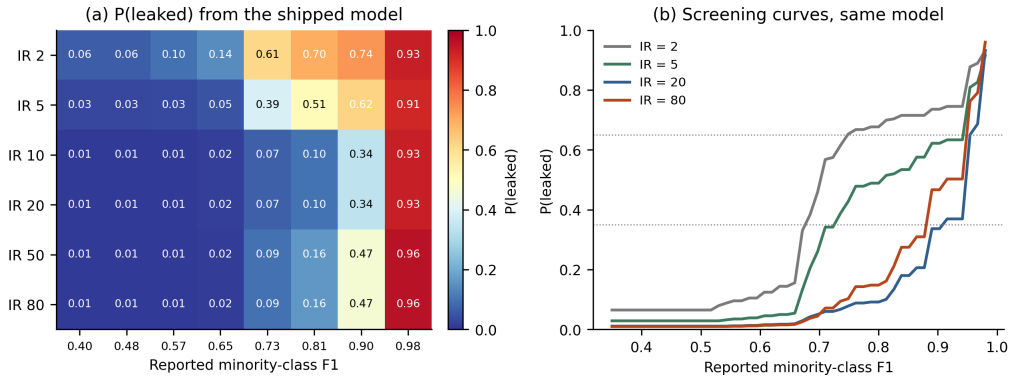


Figure 7: (a) Heatmap of P(leaked) over reported F1 and imbalance ratio; (b) screening curves at four imbalance ratios. Both panels are produced by the shipped NumPy walker, not by a re-fitted estimator.

a genuinely separable encoding—can produce a similar signature.

Table 3: Headline minority-class F1 in recent employee-attrition studies, screened with `leakcheck`. Protocol is what the paper itself states about where resampling sits relative to the split, not what the screen infers. "Not stated" means the ordering is not described, not that it is incorrect. Sources appear in the reference list.

Study	Dataset	Model	Reported F1	Stated protocol	P(leaked)
Hamja et al. (2025)	IBM HR	GradientBoosting	0.990	not stated	0.96
Nassreddine et al. (2026)	IBM HR	Ensemble (ROS)	0.977	not stated	0.93
Baydili & Tasci (2025)	IBM HR	GAN-Transformer	0.916	before-split	0.89
AL-Ali et al. (2026)	IBM HR	AdaBoost	0.797	not stated	0.78
Alyousef et al. (2026)	IBM HR	LogisticRegression	0.504	train-only	0.06
Cavescu & Popescu (2026)	IBM HR	LogisticRegression	0.439	train-only	0.06
AL-Ali et al. (2026)	Job change	HistGradientBoosting	0.659	not stated	0.01

7 Design decisions worth stating

7.1 The model ships as JSON, not as a pickle

A fitted scikit-learn estimator serialised with pickle can couple deployment to particular library versions and is less transparent than a plain-text representation. The screening model is instead exported as JSON—thresholds, split features and leaf values—and evaluated by a short pure-NumPy tree walker included in the package. This design keeps the run-time dependency set small, permits direct inspection of the stored model, and reduces one source of version-dependent variation. A stored fixture set checks that the NumPy evaluator reproduces the original scikit-learn predictions to within 10^{-9} in the tested environment, and `leakcheck.selftest`() runs that check on an installed copy.

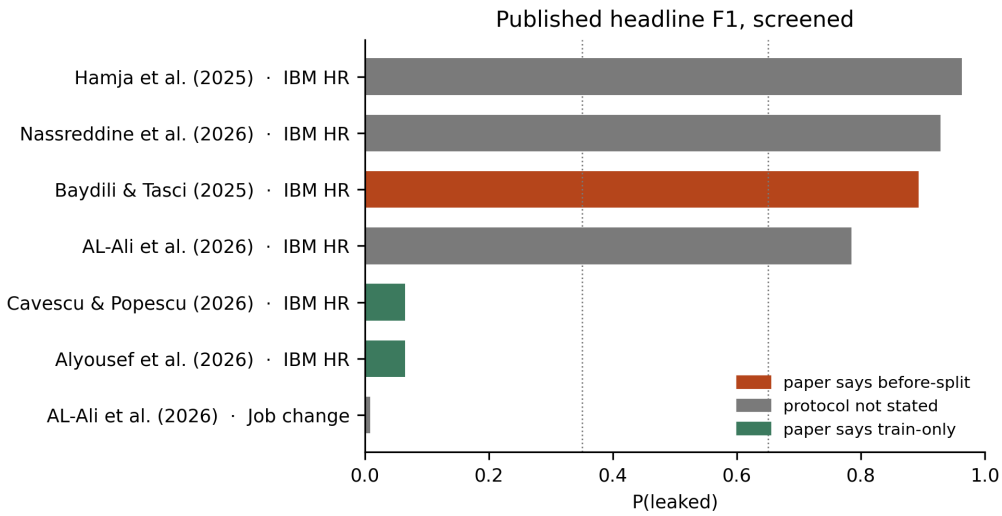


Figure 8: Published headline F1 values from Table 3, screened. Colour records what each paper stated about its own protocol. Dotted lines are the package thresholds 0.35 and 0.65.

7.2 Unknown model families resolve rather than fail

Papers name their models in many ways. The package maps around thirty spellings onto the seven families in the training corpus, and an unrecognised name falls back to the tree-ensemble family with the fallback documented, rather than raising. The purpose is a tool that gives a usable answer on real bibliographic input.

7.3 Extrapolation is announced

Inputs outside the fitted ranges—imbalance ratios beyond 1.02 to 109.9, or minority counts outside 30 to 1813—still return a model output, but the result carries an explicit note that the answer is an extrapolation. This warning is intended to reduce unwarranted confidence outside the region represented in the training corpus.

7.4 The output is a question, not a verdict

No verdict string asserts that an error occurred; the strongest package label is “likely leaked”, and the accompanying explanation recommends asking where the resampler sits. Several non-leakage explanations can produce the same signature, including a macro-averaged F1 reported as a minority-class F1, unusual preprocessing, or a genuinely separable dataset. Stronger language would exceed what can be inferred from the reported summary quantities alone.

8 Limitations and threats to validity

8.1 The signature is not causally unique

`leakcheck` is trained to recognise the empirical signature of resampling before splitting; it does not observe the pipeline that produced a reported score. Other mechanisms can generate the same combination of high F1 and apparently difficult dataset descriptors. These include target or proxy leakage, duplicate or related records crossing the split, feature construction using future information, hyperparameter selection against the test set, an unusually separable dataset, and reporting a macro-averaged or weighted F1 as though it were the minority-class F1. A high screening probability therefore supports a request for methodological clarification, not an attribution of cause.

Conversely, resampling-before-splitting need not always produce a conspicuously high score. Weak classifiers, noisy labels, severe covariate shift, or synthetic points that fail to bridge the decision boundary can yield false negatives. The screen should consequently be evaluated as a triage instrument with both false positives and false negatives, rather than as a deterministic leakage detector.

8.2 Generalisability and reported-input uncertainty

The fitted domain covers the ranges and model families in Table 1. Results outside those ranges, results from unrepresented data modalities, and results produced by substantially different resamplers are extrapolations. The model also treats quantities transcribed from a paper as if they were measured without error, although minority counts, encoded feature counts, model-family labels, and even the meaning of "F1" may be ambiguous. The fallback for an unknown classifier family improves usability but introduces additional model uncertainty that is not represented in the returned number.

The probability is conditional on the balanced controlled corpus, in which leaked and honest records are created in matched pairs. Its numerical value is not automatically the positive predictive value expected in a field where the prevalence of resampling-before-splitting is unknown. The verdict thresholds are therefore descriptive review bands rather than universal diagnostic cut-offs.

8.3 External validation

Table 3 is an application example, not an independent estimate of diagnostic performance. Its protocol labels are based on what the papers state, and "not stated" is not a ground-truth leakage label. A definitive external validation would require a preregistered literature search, duplicate blinded extraction, adjudication of disagreements, access to confirmed pipelines where possible, and a test corpus that does not overlap the training datasets. Until such validation is available, sensitivity, specificity, and predictive values should not be inferred from Table 3.

9 State of the field

Existing tooling for leakage generally operates on code or data. Validation libraries can detect some forms of train/test contamination when they can inspect both partitions. Reproducibility checklists [3, 47] support better practice but do not by themselves evaluate a number already in print. Splitters such as DataSAIL [64] can reduce specific leakage risks when the analyst has access to the data. Pipeline objects in imbalanced-learn [5] facilitate the correct prospective ordering of resampling and splitting. `leakcheck` addresses a different use case: retrospective screening from published summary information, with the limitations described above.

10 Quality control

The test suite covers input validation, model-family alias resolution, selected checks of probability behaviour as the reported score changes, agreement between batched and single-record evaluation, the command line interface, and stored regression fixtures. The package is installable from source with a standard build backend and carries a machine-readable citation file.

Availability

Source, tests, documentation and the exported screening model are released under the BSD 3-Clause licence. The repository and archival record are `occbuu/leakcheck`, archived on Zenodo. The corpus on which the screening model was trained is described in the companion methodological study, released alongside it.

Data Availability

The source code and associated materials are available from the `occbuu/leakcheck` repository and are archived on Zenodo at DOI: 10.5281/zenodo.22196497.

A Training corpus

Table 4 lists the 41 public datasets on which the screening model was trained. Six are employee-attrition datasets; the rest are Penn Machine Learning Benchmarks covering medicine, biology, finance, computer vision and several other applied fields. Imbalance ratios run from 1.02 to 109.9.

Table 4: Training corpus for `leakcheck`. Field is the application area, not the experimental split (HR vs benchmark) used in the companion study. n is the number of rows after the shared preprocessing; IR is majority / minority.

Dataset	Field	n	IR
dna	Biology	3,186	3.16
ecoli	Biology	327	8.34
yeast	Biology	1,479	48.30
adult	Census	5,000	3.18
clean2	Chemistry	5,000	5.49
wine_quality_white	Chemistry	4,898	29.05
segmentation	Computer vision	2,310	6.00
mfeat_karhunen	Computer vision	2,000	9.00
optdigits	Computer vision	5,000	9.14
satimage	Computer vision	5,000	9.29
pendigits	Computer vision	5,000	9.42
letter	Computer vision	5,000	26.17
titanic	Demography	2,207	2.10
contraceptive_method	Demography	1,473	3.42
credit_approval_australia	Finance	690	1.25
credit_approval_germany	Finance	1,000	2.33
coil2000	Finance	5,000	15.78
tic_tac_toe	Games	958	1.89
car_evaluation	Games	1,728	25.58
turnover_russia	HR	1,129	1.02
saudi_attrition	HR	1,191	1.31
job_change_ds	HR	5,000	3.01
hr_case_study	HR	4,410	5.20
ibm_hr	HR	1,470	5.20
healthcare_attr	HR	1,676	7.42
churn	Marketing	5,000	6.07
bupa	Medical	345	1.04
haberman	Medical	306	2.78
dermatology	Medical	366	6.47
hypothyroid	Medical	3,163	19.95
ann_thyroid	Medical	5,000	42.48
allhyper	Medical	3,771	46.73
dis	Medical	3,772	64.03
allrep	Medical	3,772	109.94
sonar	Physics	208	1.14
ionosphere	Physics	351	1.79
magic	Physics	5,000	1.84
phoneme	Speech	5,000	2.41
vowel	Speech	990	10.00
spambase	Text	4,601	1.54
analcatdata_authorship	Text	841	14.29

References

[1] Chawla, N.V., Bowyer, K.W., Hall, L.O., and Kegelmeyer, W.P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357.

[2] Kaufman, S., Rosset, S., Perlich, C., and Stitelman, O. (2012). Leakage in data mining: formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4), 15:1–21.

[3] Kapoor, S. and Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9), 100804.

[4] Vandewiele, G., Dehaene, I., Kovacs, G., Sterckx, L., Janssens, O., Ongenae, F., De Backere, F., De Turck, F., Roelens, K., Decruyenaere, J., Van Hoecke, S., and Demeester, T. (2021). Overly optimistic prediction results on imbalanced data: a case study of flaws and benefits when applying over-sampling. *Artificial Intelligence in Medicine*, 111, 101987.

[5] Lemaitre, G., Nogueira, F., and Aridas, C.K. (2017). Imbalanced-learn: a Python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research*, 18(17), 1–5.

[6] Pineau, J., Vincent-Lamarre, P., Sinha, K., Larivière, V., Beygelzimer, A., d'Alché-Buc, F., Fox, E., and Larochelle, H. (2021). Improving reproducibility in machine learning research: a report from the NeurIPS 2019 reproducibility program. *Journal of Machine Learning Research*, 22(164), 1–20.

[7] Bouthillier, X., Delaunay, P., Bronzi, M., Trofimov, A., Nichyporuk, B., Szeto, J., Sepah, N., Raff, E., Madan, K., Voleti, V., Kahou, S.E., Michalski, V., Arbel, T., and Vincent, P. (2021). Accounting for variance in machine learning benchmarks. In *Proceedings of the 38th International Conference on Machine Learning*, 7479–7490.

[8] Varoquaux, G. and Cheplygina, V. (2022). Machine learning for medical imaging: methodological failures and recommendations for the future. *npj Digital Medicine*, 5, 48.

[9] Lipton, Z.C. and Steinhardt, J. (2018). Troubling trends in machine learning scholarship. *arXiv preprint arXiv:1807.03341*.

[10] Roberts, M. et al. (2021). Common pitfalls and recommendations for using machine learning to detect and prognosticate for COVID-19 using chest radiographs and CT scans. *Nature Machine Intelligence*, 3, 199–217.

[11] Sculley, D. et al. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems*, 28, 2503–2511.

[12] Dacrema, M.F., Cremonesi, P., and Jannach, D. (2019). Are we really making much progress? A worrying analysis of recent neural recommendation approaches. In *Proceedings of the 13th ACM Conference on Recommender Systems*, 101–109.

[13] Musgrave, K., Belongie, S., and Lim, S.-N. (2020). A metric learning reality check. In *Computer Vision–ECCV 2020*, 681–699.

[14] He, H. and Garcia, E.A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263–1284.

[15] Batista, G.E.A.P.A., Prati, R.C., and Monard, M.C. (2004). A study of the behavior of several methods for balancing machine learning training data. *SIGKDD Explorations Newsletter*, 6(1), 20–29.

[16] Han, H., Wang, W.-Y., and Mao, B.-H. (2005). Borderline-SMOTE: a new over-sampling method in imbalanced data sets learning. In *Advances in Intelligent Computing*, 878–887.

[17] He, H., Bai, Y., Garcia, E.A., and Li, S. (2008). ADASYN: adaptive synthetic sampling approach for imbalanced learning. In *2008 IEEE International Joint Conference on Neural Networks*, 1322–1328.

[18] Fernández, A., García, S., Galar, M., Prati, R.C., Krawczyk, B., and Herrera, F. (2018). Learning from imbalanced data sets. Springer.

[19] Blagus, R. and Lusa, L. (2013). SMOTE for high-dimensional class-imbalanced data. *BMC Bioinformatics*, 14, 106.

[20] Elreedy, D. and Atiya, A.F. (2019). A comprehensive analysis of synthetic minority oversampling technique (SMOTE) for handling class imbalance. *Information Sciences*, 505, 32–64.

[21] García, V., Sánchez, J.S., and Mollineda, R.A. (2012). On the effectiveness of preprocessing methods when dealing with different levels of class imbalance. *Knowledge-Based Systems*, 25(1), 13–21.

[22] Branco, P., Torgo, L., and Ribeiro, R.P. (2016). A survey of predictive modeling on imbalanced domains. *ACM Computing Surveys*, 49(2), Article 31.

[23] Krawczyk, B. (2016). Learning from imbalanced data: open challenges and future directions. *Progress in Artificial Intelligence*, 5, 221–232.

[24] Buda, M., Maki, A., and Mazurowski, M.A. (2018). A systematic study of the class imbalance problem in convolutional neural networks. *Neural Networks*, 106, 249–259.

[25] van den Goorbergh, R., van Smeden, M., Timmerman, D., and Van Calster, B. (2022). The harm of class imbalance corrections for risk prediction models: illustration and simulation using logistic regression. *Journal of the American Medical Informatics Association*, 29(9), 1525–1534.

[26] Saito, T. and Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE*, 10(3), e0118432.

[27] Tambe, P., Cappelli, P., and Yakubovich, V. (2019). Artificial intelligence in human resources management: challenges and a path forward. *California Management Review*, 61(4), 15–42.

[28] Fallucchi, F., Coladangelo, M., Giuliano, R., and De Luca, E.W. (2020). Predicting employee attrition using machine learning techniques. *Computers*, 9(4), 86.

[29] Rombaut, E. and Guerry, M.-A. (2018). Predicting voluntary turnover through human resources database analysis. *Management Research Review*, 41(1), 96–112.

[30] Strohmeier, S. (2020). Digital human resource management: a conceptual clarification. *German Journal of Human Resource Management*, 34(3), 345–365.

[31] Köchling, A. and Wehner, M.C. (2020). Discriminated by an algorithm: a systematic review of discrimination and fairness by algorithmic decision-making in the context of HR recruitment and HR development. *Business Research*, 13, 795–848.

[32] Chalfin, A., Danieli, O., Hillis, A., Jelveh, Z., Luca, M., Ludwig, J., and Mullainathan, S. (2016). Productivity and selection of human capital with machine learning. *American Economic Review*, 106(5), 124–127.

[33] Marler, J.H. and Boudreau, J.W. (2017). An evidence-based review of HR analytics. *The International Journal of Human Resource Management*, 28(1), 3–26.

[34] Lundberg, S.M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, 30, 4765–4774.

[35] Lundberg, S.M. et al. (2020). From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2, 56–67.

[36] Ribeiro, M.T., Singh, S., and Guestrin, C. (2016). Why should I trust you? Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1135–1144.

[37] Chen, T. and Guestrin, C. (2016). XGBoost: a scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.

[38] Molnar, C. (2022). *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable* (2nd ed.).

[39] Doshi-Velez, F. and Kim, B. (2017). Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608*.

[40] Rudin, C. (2019). Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1, 206–215.

[41] Barredo Arrieta, A. et al. (2020). Explainable Artificial Intelligence (XAI): concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion*, 58, 82–115.

[42] Niculescu-Mizil, A. and Caruana, R. (2005). Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning*, 625–632.

[43] Guo, C., Pleiss, G., Sun, Y., and Weinberger, K.Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, 1321–1330.

[44] Davis, J. and Goadrich, M. (2006). The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning*, 233–240.

[45] Hand, D.J. (2009). Measuring classifier performance: a coherent alternative to the area under the ROC curve. *Machine Learning*, 77, 103–123.

[46] Apicella, A., Isgro, F., and Prevete, R. (2025). Don't push the button! Exploring data leakage risks in machine learning and transfer learning. *Artificial Intelligence Review*, 58, 339.

[47] Kapoor, S., Cantrell, E., Peng, K., Pham, T.H., Bail, C.A., Gundersen, O.E., Hofman, J.M., Hullman, J., Lones, M.A., Malik, M.M., Nanayakkara, P., Poldrack, R.A., Raji, I.D., Roberts, M., Salganik, M.J., Serra-Garcia, M., Stewart, B.M., Vandewiele, G., and Narayanan, A. (2024). REFORMS: consensus-based recommendations for machine-learning-based science. *Science Advances*, 10(18), eadk3452.

[48] Semmelrock, H., Ross-Hellauer, T., Kopeinik, S., Theiler, D., Haberl, A., Thalmann, S., and Kowald, D. (2025). Reproducibility in machine-learning-based research: overview, barriers, and drivers. *AI Magazine*, 46(2), e70002.

[49] Lones, M.A. (2024). Avoiding common machine learning pitfalls. *Patterns*, 5(10), 101046.

[50] Elor, Y. and Averbuch-Elor, H. (2022). To SMOTE, or not to SMOTE? *arXiv preprint arXiv:2201.08528*.

[51] Elreedy, D., Atiya, A.F., and Kamalov, F. (2024). A theoretical distribution analysis of synthetic minority oversampling technique (SMOTE) for imbalanced learning. *Machine Learning*, 113(7), 4903–4923.

[52] Carriero, A., Luijken, K., de Hond, A., Moons, K.G.M., Van Calster, B., and van Smeden, M. (2025). The harms of class imbalance corrections for machine learning based prediction models: a simulation study. *Statistics in Medicine*, 44(3–4), e10320.

[53] Longo, L. et al. (2024). Explainable Artificial Intelligence (XAI) 2.0: a manifesto of open challenges and interdisciplinary research directions. *Information Fusion*, 106, 102301.

[54] Giermindl, L.M., Strich, F., Christ, O., Leicht-Deobald, U., and Redzepi, A. (2022). The dark sides of people analytics: reviewing the perils for organisations and employees. *European Journal of Information Systems*, 31(3), 410–431.

[55] Budhwar, P. et al. (2023). Human resource management in the age of generative artificial intelligence: perspectives and research agenda on ChatGPT, Bard, and beyond. *Human Resource Management Journal*, 33(3), 606–659.

[56] Bandara, R.J., Biswas, K., Akter, S., Shafique, S., and Rahman, M. (2025). Addressing algorithmic bias in AI-driven HRM systems: implications for strategic HRM effectiveness. *Human Resource Management Journal*, 35(4), 1047–1063.

[57] Chen, Z. (2026). Algorithmic human resource management as a mode of algorithmic governance: transparency, fairness, and human agency in the digital workplace. *Humanities and Social Sciences Communications*, 13, 594.

[58] Cavescu, A.M. and Popescu, A.N. (2026). Leakage-free evaluation for employee attrition prediction on tabular data. *Information*, 17(3), 308.

[59] Hamja, A., Hasan, M., Hassan, Z., Uddin, M.P., and Siddikee, M.J.A. (2025). An explainable machine learning-based employee attrition predictive system. *Annals of Data Science*, doi:10.1007/s40745-025-00617-9.

[60] Nassreddine, G., Hammoud, J., Al-Khatib, O., and Al Majzoub, M. (2026). Employee attrition prediction: an explanatory and statistically robust ensemble learning model. *Computers*, 15(3), 185.

[61] Baydili, I.T. and Tasci, B. (2025). Predicting employee attrition: XAI-powered models for managerial decision-making. *Systems*, 13(7), 583.

[62] AL-Ali, M., Alwateer, M., Alsaedi, S.A., Balaha, H.M., Badawy, M., and Elhosseini, M.A. (2026). Integrating machine learning and explainable AI for employee attrition prediction in HR analytics. *Scientific Reports*, 16, 6344.

[63] Alyousef, M.I., Khan, H.W., and Sattar, M.U. (2026). A hybrid predictive model for employee turnover: integrating ensemble learning and feature-driven insights from IBM HR Analytics. *Information*, 17(2), 208.

[64] Joeres, R., Blumenthal, D.B., and Kalinina, O.V. (2025). Data splitting to avoid information leakage with DataSAIL. *Nature Communications*, 16, 3337.

[65] Olson, R.S., La Cava, W., Orzechowski, P., Urbanowicz, R.J., and Moore, J.H. (2017). PMLB: a large benchmark suite for machine learning evaluation and comparison. *BioData Mining*, 10, 36.

[66] Brier, G.W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1), 1–3.